Educational Lecture Notes: Understanding the Smart Secure Email Codebase

Secure Email Project

April 11, 2026

Contents

1 Purpose of This Lecture **2**

2 What This Project Builds **2**

3 The Big Design Idea **3**

4 Repository Map **4**

4.1 1 Top Level 4

4.2 2 Why This Layout Matters 4

5 How to Read This Codebase **5**

6 Shared Layer: common/ **6**

7 common/config.py: Domain Configuration and Storage Isolation **6**

7.1 1 Why This File Is Important 6

7.2 2 from_mapping(...) and from_file(...) 7

7.3 3 ensure_layout() 7

8 common/crypto.py: Passwords, HMACs, and Signed Tokens **7**

8.1 1 Password Hashing 7

8.2 2 Session and Secret Generation 8

8.3 3 Hashing Attachment Content 8

8.4 4 Request and Relay MACs 8

8.5 5 Quick Action Tokens 9

9 common/schemas.py: The API Contract **9**

9.1 1 Why This File Matters 10

9.2 2 A Few Especially Important Models . . . 10

10 common/text_features.py: Lightweight Intelligence 10

10.1 1 Tokenization . . . 11

10.2 2 Keyword Extraction . . . 11

10.3 3 Classification . . . 11

10.4 4 Phishing Heuristics . . . 11

10.5 5 Quick Reply Suggestions . . . 12

10.6 6 Fuzzy Matching . . . 12

11 A Note on common/utils.py 12

12 Server Bootstrap: server/main.py 13

12.1 1 What create_app(...) Does . . . 13

12.2 2 Why This File Is Clean . . . 13

12.3 3 main() . . . 14

13 server/storage.py: The Persistent Heart of the System 14

13.1 1 Why AppContext Exists . . . 14

13.2 2 Database Choice . . . 15

13.3 3 Core Tables and Their Meaning . . . 15

13.4 4 Why mail_items Is Designed This Way . . . 15

13.5 5 Queue Support Methods . . . 16

13.6 6 Relay Helpers . . . 16

13.7 7 Audit and Alert Persistence . . . 17

14 server/auth.py: Authentication and Post-Login Integrity 17

14.1 1 register(...) . . . 17

14.2 2 login(...) . . . 17

14.3 3 get_current_user(...) . . . 18

14.4 4 verify_authenticated_request(...) . . . 18

14.5 5 Why This Matters . . . 19

15 server/rate_limit.py: Abuse Prevention 19

15.1 1 Sliding Window Budgeting . . . 19

15.2 2 Login Protection . . . 20

15.3 3 Send and Upload Limits . . . 20

15.4 4 Audit and Alert Integration . . . 20

16 server/attachments.py: Safe Image Attachments **20**
16.1 1 Allowed Types . . . 21
16.2 2 Content Validation . . . 21
16.3 3 Deduplicated Blob Storage . . . 21
16.4 4 Authorization on Download . . . 21
16.5 5 Cross-Domain Attachment Transfer . . . 22

17 server/phishing.py: Thin Wrapper, Clear Responsibility **22**

18 server/mailbox.py: Main Business Logic **22**
18.1 1 Helper Functions . . . 22
18.2 2 store_mail_copy(...) . . . 23
18.3 3 Suspicious Mail Handling . . . 23
18.4 4 Sender-Side Delivery State . . . 23
18.5 5 dispatch_message(...) . . . 24
18.6 6 Draft Handling . . . 24
18.7 7 Recall Logic . . . 24
18.8 8 Group Operations . . . 25
18.9 9 Quick Actions . . . 25
18.10 10 Search and Autocomplete . . . 25

19 server/relay.py: Trusted Cross-Domain Delivery **25**
19.1 1 Relay Verification . . . 25
19.2 2 Incoming Relay . . . 26
19.3 3 Relay Recall . . . 26

20 server/workers.py: Background Delivery Runtime **26**
20.1 1 Worker Startup . . . 27
20.2 2 Worker Loop . . . 27
20.3 3 Job Type Routing . . . 27
20.4 4 Local Delivery . . . 27
20.5 5 Remote Delivery . . . 27
20.6 6 Inbound Delivery . . . 28
20.7 7 Why This File Matters So Much . . . 28

21 server/logging.py: Structured Audit and Alerts **28**

22 server/web.py: Serving the Browser Frontend **29**

23 Client Layer: client/ **29**

24 client/api.py: Session Store and Signed API Calls **29**

24.1 1 SessionState 29

24.2 2 SessionStore 30

24.3 3 ApiClient.register(...) 30

24.4 4 ApiClient.login(...) 30

24.5 5 _auth_headers(...) 30

24.6 6 API Convenience Methods 30

25 client/cli.py, client/ui.py, and client/quick_reply.py **31**

26 Browser Frontend: web/index.html **31**

27 Browser Frontend Logic: web/app.js **32**

27.1 1 State Model 32

27.2 2 Session Persistence 32

27.3 3 Register and Login 32

27.4 4 Signed Browser Requests 33

27.5 5 Client-Side Anti-Abuse Guard 33

27.6 6 Rendering Safety 33

27.7 7 Detail View 33

28 End-to-End Flow: Registration **34**

28.1 1 Browser or CLI Side 34

28.2 2 Server Side 34

28.3 3 Storage Side 34

29 End-to-End Flow: Login **34**

29.1 1 Client Side 34

29.2 2 Server Side 35

29.3 3 Result 35

30 End-to-End Flow: Signed Authenticated POST **35**

30.1 1 Client Builds Headers 35

30.2 2 Server Verifies Them 36

30.3 3 Why This Is Better Than a Plain Session Cookie 36

31 End-to-End Flow: Local Mail Send **36**
31.1 1 Request Handling . . . 36
31.2 2 Dispatch . . . 36
31.3 3 Worker Completion . . . 37
31.4 4 Delivery State . . . 37

32 End-to-End Flow: Cross-Domain Mail Send **37**
32.1 1 On Domain A . . . 37
32.2 2 Delivery Worker on Domain A . . . 37
32.3 3 On Domain B . . . 38
32.4 4 Inbound Worker on Domain B . . . 38

33 End-to-End Flow: Recall **38**
33.1 1 Sender Requests Recall . . . 38
33.2 2 Local Validation . . . 38
33.3 3 Queue-Aware Recall . . . 39
33.4 4 Delivered Recall . . . 39
33.5 5 Remote Recall . . . 39
33.6 6 Why This Design Is Good . . . 39

34 End-to-End Flow: Quick Actions **39**
34.1 1 Token Creation . . . 39
34.2 2 Token Execution . . . 39

35 End-to-End Flow: Search and Smart Suggestions **40**

36 Data Model Deep Dive **40**
36.1 1 Users and Sessions . . . 40
36.2 2 Replay Defense Tables . . . 40
36.3 3 Mail and Attachments . . . 41
36.4 4 Productivity and Smart UX . . . 41
36.5 5 Asynchronous Runtime . . . 41

37 Browser UI as a Security Teaching Tool **41**

38 Testing Philosophy **41**
38.1 1 Test Fixture Setup . . . 42
38.2 2 _signed_headers(...) . . . 42

38.3 3 _relay(...) . . . 42

39 Walkthrough of Key Tests **42**

39.1 1 test_login_lockout . . . 42

39.2 2 test_register_requires_matching_confirmation . . . 42

39.3 3 test_web_root_is_served . . . 42

39.4 4 test_cross_domain_send_attachment_recall_and_tamper . . . 42

39.5 5 test_phishing_sample_is_flagged . . . 43

39.6 6 test_attachment_dedup_reuses_single_blob . . . 43

39.7 7 test_cc_recipients_are_delivered . . . 43

39.8 8 test_concurrent_multi_client_send_and_receive . . . 43

39.9 9 test_replay_rejected . . . 43

39.1010 test_invalid_attachment_rejected . . . 43

39.1111 test_send_rate_limit . . . 43

40 Security Model Mapped Directly to Code **43**

40.1 1 Password Theft Risk . . . 44

40.2 2 Brute-Force Login . . . 44

40.3 3 Tampered Authenticated Request . . . 44

40.4 4 Replay Attack . . . 44

40.5 5 Open Relay or Forged Relay . . . 44

40.6 6 Attachment Abuse . . . 45

40.7 7 Quick Action Forgery . . . 45

40.8 8 XSS From Malicious Mail . . . 45

41 Document Set and How It Relates to the Code **45**

41.1 1 docs/protocol.md . . . 45

41.2 2 docs/threat_model.md . . . 46

41.3 3 docs/test_report.md . . . 46

41.4 4 docs/assignment_coverage.md . . . 46

41.5 5 docs/implementation_structure.txt . . . 46

41.6 6 docs/alerting_and_audit.md . . . 46

41.7 7 docs/end_to_end_encryption_design.md . . . 47

41.8 8 docs/malicious_mail_poc.md . . . 47

41.9 9 docs/p2p_feasibility.md . . . 47

42 Bonus and Extension Topics **47**

42.1 1 End-to-End Encryption 47
42.2 2 Malicious Mail Scripts 47
42.3 3 P2P Feasibility 48

43 What Is Especially Good About This Codebase 48

43.1 1 It Is Small but Real 48
43.2 2 Security Is Not Bolted On 48
43.3 3 The Smart Features Are Bounded 49
43.4 4 The Tests Prove Real Claims 49

44 Honest Limitations 49

44.1 1 TLS in Demo Mode 49
44.2 2 SQLite Scope 49
44.3 3 Heuristic Intelligence 49
44.4 4 Session Simplification 49
44.5 5 Queue Reliability Scope 49

45 How to Explain This Project in an Exam or Presentation 50

46 Suggested Reading Path for a New Developer 50

47 Study Questions 50

48 Final Summary 51

1 Purpose of This Lecture

This document is not just a project summary. It is a teaching-oriented code walkthrough for the entire smart secure email system. The goal is that a student, reviewer, teammate, or examiner can read this file and understand:

- what the system is supposed to do
- how the repository is organized
- how the code is divided into modules
- how the main runtime flows actually work
- where the security mechanisms live in code
- how the "smart" features are implemented
- how storage, relay, workers, and testing fit together
- what was implemented directly and what remains a design extension

This lecture is intentionally broader than `README.md`. The README gives the project story. This lecture explains the codebase that realizes that story. It also connects the written documents in `docs/` back to the source files in `common/`, `server/`, `client/`, `web/`, `scripts/`, and `tests/`.

If you only want a quick introduction, start with:

1. `README.md`
2. `server/main.py`
3. `server/auth.py`
4. `server/storage.py`
5. `server/mailbox.py`
6. `server/relay.py`
7. `server/workers.py`
8. `web/app.js`
9. `tests/test_secure_mail.py`

If you want to deeply understand the implementation, read this lecture from start to finish.

2 What This Project Builds

The project implements a local "smart secure email" prototype with two isolated mail domains. It does not rely on a mature mail server such as Postfix, Dovecot, Exim, or Exchange as its core runtime. Instead, it builds a small mail system over HTTP and JSON, with:

- two separate domain servers

- two separate local storage roots
- client registration and login
- inbox, sent, and draft folders
- cross-domain mail delivery
- group send and saved groups
- image attachments
- recall of unread mail within a limited window
- quick reply suggestions
- quick actions such as add-TODO and acknowledge
- fuzzy search and contact autocomplete
- phishing heuristics
- rate limiting and login lockout
- audit logging and alert logging
- a queue-backed worker model
- automated tests
- a browser frontend and a CLI frontend

This is important because the assignment is not asking for a toy CRUD app. It asks for a usable, testable, security-aware email system with basic intelligence and distributed behavior across isolated domains.

3 The Big Design Idea

The most important architectural idea in this repository is:

Request handlers do short, security-critical work quickly, and slower work is moved into queued background processing.

That is why the code does not put all delivery work directly inside the send request. Instead, the send request:

1. authenticates the user
2. verifies request integrity
3. validates recipients and attachments
4. writes sender-visible mailbox state
5. enqueues local and remote delivery jobs
6. returns a fast response such as `queued`

Then background worker threads process the slower parts:

- local inbox insertion
- cross-domain relay
- inbound delivery from another domain
- retries on failure
- delivery state refresh

This design lets the project show concurrency thinking, failure handling, and clear separation of concerns.

4 Repository Map

The repository is intentionally split into layers.

4.1 1 Top Level

`README.md` Project story, architecture overview, security goals, concurrency model, and demo direction.

`pyproject.toml` Dependencies, editable install metadata, pytest setup, and package config.

`common/` Shared code used by both client and server.

`server/` Backend implementation for one domain server.

`client/` CLI client and its session/signing logic.

`web/` Browser frontend assets.

`configs/` Dual-domain example configuration.

`scripts/` Startup, demo, and stress-test scripts.

`docs/` Protocol, threat model, testing, assignment mapping, and extension docs.

`tests/` Automated tests for functional, security, and concurrency-sensitive behavior.

4.2 2 Why This Layout Matters

The layout is educationally useful because each folder corresponds to a responsibility boundary:

- `common` defines shared contracts and cryptographic helpers
- `server` implements trusted domain logic
- `client` implements authenticated caller behavior
- `web` implements the browser user interface
- `tests` acts as executable proof of claims made in the docs

This keeps the project explainable.

5 How to Read This Codebase

A lot of confusion in code reviews comes from reading files in the wrong order. For this project, the best reading order is:

1. `common/config.py`
2. `common/schemas.py`
3. `common/crypto.py`
4. `server/main.py`
5. `server/storage.py`
6. `server/auth.py`
7. `server/rate_limit.py`
8. `server/attachments.py`
9. `server/mailbox.py`
10. `server/relay.py`
11. `server/workers.py`
12. `client/api.py`
13. `web/app.js`
14. `tests/test_secure_mail.py`

Why this order?

- `config.py` tells you how a domain is configured.
- `schemas.py` tells you the shape of the data.
- `crypto.py` tells you how passwords, HMACs, and action tokens work.
- `main.py` shows how everything is wired together.
- `storage.py` reveals the persistent data model.
- `auth.py` explains who is allowed to do what.
- `mailbox.py` is the main business logic.
- `relay.py` and `workers.py` explain the distributed behavior.
- `client/api.py` and `web/app.js` show how requests are signed.
- `tests/test_secure_mail.py` proves which claims are actually verified.

6 Shared Layer: common/

The `common/` package contains the shared language of the system. This is where the project defines reusable concepts instead of duplicating them across server and client code.

7 common/config.py: Domain Configuration and Storage Isolation

The file `common/config.py` defines `DomainConfig`, a dataclass that describes one mail domain instance.

Important fields include:

- `domain`
- `data_root`
- `peer_domains`
- `host`
- `port`
- `session_ttl_minutes`
- `recall_window_minutes`
- `login_max_attempts`
- `login_window_seconds`
- `lockout_seconds`
- `send_rate_limit_per_minute`
- `upload_rate_limit_bytes_per_minute`
- `max_attachment_bytes`
- `action_secret`
- `relay_secret`
- `ssl_certfile`
- `ssl_keyfile`

7.1 1 Why This File Is Important

This file is where configuration stops being "just settings" and becomes part of the security model.

For example:

- `data_root` enforces per-domain storage isolation

- `peer_domains` defines the trust boundary for relay
- `action_secret` signs quick-action tokens
- `relay_secret` authenticates server-to-server relay
- lockout and rate limit values shape abuse resistance
- recall windows define business logic for message recall

7.2 2 from_mapping(...) and from_file(...)

`DomainConfig.from_mapping(...)` converts raw YAML or dict values into a typed config object. It also:

- resolves `data_root` to an absolute path
- copies unknown values into `extra`
- applies defaults
- calls `ensure_layout()`

`from_file(...)` simply reads YAML and passes it to `from_mapping(...)`.

7.3 3 ensure_layout()

The method creates a predictable domain-local directory structure:

- `<data_root>/users`
- `<data_root>/mail`
- `<data_root>/attachments/blobs`
- `<data_root>/logs`

This is one of the cleanest demonstrations of domain isolation in the code. Server A and Server B do not share the same SQLite file, blob directory, or log directory.

8 common/crypto.py: Passwords, HMACs, and Signed Tokens

This file is central to the security design.

8.1 1 Password Hashing

The project uses Argon2id through `argon2-cffi`.

```
PASSWORD_HASHER=PasswordHasher(...)
```

The chosen settings are modest enough for a local demo but still reflect a real password hashing strategy:

- non-plaintext password storage
- slow verification relative to simple hashes
- salts handled by the library

The key functions are:

- `hash_password(password)`
- `verify_password(password_hash,password)`

This means passwords are not stored in plaintext in the `users` table.

8.2 2 Session and Secret Generation

`new_session_token()` uses `secrets.token_urlsafe(32)`.

This is used for:

- session tokens
- session MAC keys

That means the post-login integrity system has a cryptographically random secret per session.

8.3 3 Hashing Attachment Content

`sha256_hex(data)` computes the content hash of attachment bytes.

This hash becomes the deduplication key for attachment blobs. The system stores logical attachments separately from physical blobs.

8.4 4 Request and Relay MACs

`mac_hex(secret,message)` computes an HMAC-SHA256 hex digest.

This function is reused in two places:

- post-login client-to-server request signing
- server-to-server relay signing

This is a very important design choice. The project does not treat "logged in" as enough for all future actions. Instead, sensitive state-changing requests are individually authenticated with a MAC over canonical request content.

8.5 5 Quick Action Tokens

`sign_payload(secret,payload)` and `verify_signed_payload(secret,token)` are used to build safe action buttons inside email details.

Instead of trusting the browser to say "run add_todo for this message", the server gives the client a signed token that encodes:

- the message ID
- the recipient
- the action type
- the action title

When the client sends it back, the server verifies the signature. This is a stronger design than trusting raw action parameters from the UI.

9 common/schemas.py: The API Contract

This file contains Pydantic models that define the shapes of requests and responses.

Examples:

- `RegisterRequest`
- `LoginRequest`
- `AuthResponse`
- `AttachmentUploadRequest`
- `AttachmentMeta`
- `MailSummary`
- `SendMailRequest`
- `DraftRequest`
- `RecallRequest`
- `GroupCreateRequest`
- `GroupMemberRequest`
- `GroupSendRequest`
- `ActionExecutionRequest`
- `TodoItem`
- `ContactSuggestion`
- `SearchResponse`
- `RelayIncomingRequest`
- `RelayRecallRequest`

9.1 1 Why This File Matters

This is the language boundary between modules.

When you read `server/mailbox.py`, for example, you can understand its inputs because the request shapes are already formalized in `common/schemas.py`.

9.2 2 A Few Especially Important Models

`RegisterRequest` Includes `confirm_password`, which was added so the registration flow matches the assignment more strictly.

`MailSummary` This is the main read-model for a message in the UI. It includes:

- IDs
- folder
- delivery state
- sender and recipients
- attachments
- security flags
- keywords
- classification
- quick replies
- quick actions
- recall state
- read state

This shows that mailbox rows are not just raw mail text. They carry smart and security metadata too.

`RelayIncomingRequest` This is the cross-domain mail envelope. It includes sender, recipients, thread ID, subject, body, timestamp, and attachments.

10 common/text_features.py: Lightweight Intelligence

This file implements the "smart" layer. It is intentionally simple and local. There is no external LLM call and no heavy ML pipeline. That is a strength in a teaching project because the logic is visible and testable.

10.1 1 Tokenization

`tokenize(text)` uses a regex to pull tokens and remove stopwords.

This supports:

- keyword extraction
- classification
- fuzzy search input preparation

10.2 2 Keyword Extraction

`extract_keywords(text,corpus,top_k=5)` is a small TF-IDF-like scorer:

- count tokens in the current message
- estimate document frequency from a recent local corpus
- combine term frequency and inverse document frequency
- return top keywords

This gives the project one "algorithmic enhancement" without requiring an external model.

10.3 3 Classification

`classify_message(...)` uses rule-based labels:

- Finance
- HR
- Scheduling
- Security
- Support
- General

This is a simple but understandable classifier. Educationally, this is useful because the reader can see exactly why a message becomes "Security" or "Scheduling".

10.4 4 Phishing Heuristics

`phishing_flags(...)` computes:

- `phishing_score`
- `suspicious`

- `reasons`

It looks for:

- urgent or credential language
- multiple links
- reply-to domain mismatch
- action-request language

This is not a production anti-phishing engine, but it satisfies the assignment requirement for basic phishing recognition and is easy to explain in a defense.

10.5 5 Quick Reply Suggestions

`quick_reply_suggestions(...)` generates small reply templates based on the message content.

Examples of triggered logic:

- question-like text -> "Yes, that works for me."
- meeting or date language -> schedule confirmation or reschedule suggestion
- gratitude -> "Thanks, received."

Again, this is advisory logic, not autonomous action.

10.6 6 Fuzzy Matching

`levenshtein(...)` and `fuzzy_score(...)` power:

- mailbox search
- contact autocomplete

That means search is not exact-string only.

11 A Note on common/utils.py

Even though this file is small, it supports the whole project by centralizing boring but important utilities:

- UTC timestamps
- ISO timestamp formatting
- parsing timestamps

- generating IDs
- normalizing email addresses
- extracting email domains
- canonical JSON dumping
- ensuring directories exist

This matters because security-sensitive systems often break not because of cryptography failure, but because of inconsistent timestamp handling, malformed IDs, or inconsistent serialization. Centralizing these helpers reduces that risk.

12 Server Bootstrap: server/main.py

`server/main.py` is the entry point for one domain server.

The main exported function is:

`create_app(config:DomainConfig,relay_dispatch:RelayDispatch|None=None)`

12.1 1 What create_app(...) Does

It:

- creates an `AppContext`
- defines a FastAPI lifespan block
- starts worker threads on startup
- stops workers on shutdown
- registers route modules
- attaches context to `app.state.ctx`

Registered route groups:

- `auth.register_routes(...)`
- `attachments.register_routes(...)`
- `mailbox.register_routes(...)`
- `relay.register_routes(...)`
- `web.register_routes(...)`

12.2 2 Why This File Is Clean

This file avoids putting business logic in the bootstrap layer. It wires modules together and leaves the actual work to those modules. That separation makes the project much easier to explain.

12.3 3 main()

The CLI entry point:

- parses `--config`
- optionally reads TLS cert and key paths
- loads `DomainConfig`
- creates the app
- runs Uvicorn

So each YAML config file corresponds to one isolated domain process.

13 server/storage.py: The Persistent Heart of the System

This is one of the most important files in the repository.

It defines:

- database schema
- `AppContext`
- relay helper methods
- queue methods
- audit and alert persistence

13.1 1 Why AppContext Exists

`AppContext` bundles domain-local state:

- config
- relay dispatch function
- database path
- audit log path
- alert log path
- worker stop event
- worker threads

This gives all route modules a shared domain context without using global variables.

13.2 2 Database Choice

The project uses SQLite with:

- `PRAGMAjournal_mode=WAL`
- `PRAGMAforeign_keys=ON`
- `check_same_thread=False`

WAL mode helps concurrent access patterns in a local prototype. SQLite is also a good teaching choice because:

- it is easy to inspect
- it requires no external server
- it still supports transactional state

13.3 3 Core Tables and Their Meaning

`users` Stores user accounts and Argon2id password hashes.

`sessions` Stores active session tokens, per-session HMAC keys, expiry, and last sequence number.

`rate_events` Stores sliding-window rate limit data.

`lockouts` Stores temporary login lockout deadlines.

`request_guards` Stores request IDs, nonces, and sequence metadata to prevent replay.

`relay_guards` Stores per-domain relay nonces to prevent duplicate relay requests.

`attachment_blobs` Stores physical attachment blob metadata. This is where deduplication happens.

`attachments` Stores logical attachment records visible to users.

`mail_items` Stores mailbox copies. This is one of the most important design decisions: mail is stored as per-owner mailbox items rather than one global message row with dynamic views.

`mail_attachment_links` Connects mailbox items to attachment IDs for download authorization.

`groups_store` Stores saved recipient groups.

`todos` Stores TODO items created from quick actions.

`contacts` Stores contact history for search and autocomplete.

`job_queue` Stores persistent background jobs for local delivery, remote delivery, and inbound delivery.

13.4 4 Why mail_items Is Designed This Way

Instead of trying to reconstruct mailbox views from one global canonical message object at read time, the project stores owner-scoped mailbox copies.

Benefits:

- easy authorization by `owner_email`
- clean inbox/sent/draft folder reads
- simple read/unread tracking
- per-user quick actions and quick replies
- local recall state updates

This is a very practical educational design.

13.5 5 Queue Support Methods

The queue-related methods are:

- `enqueue_job(...)`
- `claim_job(...)`
- `complete_job(...)`
- `fail_job(...)`
- `pending_jobs(...)`
- `wait_for_idle(...)`

These methods turn queueing from a documentation idea into real runtime code.

`fail_job(...)` is especially instructive because it:

- increments attempts
- truncates error text
- moves the job back to `pending` with exponential backoff
- eventually marks it `failed`

13.6 6 Relay Helpers

`relay_post_sync(...)` and `relay_post(...)` send authenticated relay traffic to peer domains.

They:

- look up the peer URL from `peer_domains`
- generate timestamp and nonce
- build canonical JSON
- compute relay HMAC
- send the request with relay headers

Important note:

`verify=False` is used in HTTP clients for the local demo. This is acceptable for a local educational prototype but should be replaced with real certificate verification in a stronger deployment.

13.7 7 Audit and Alert Persistence

`audit(...)` writes JSON lines to `security.jsonl`.

`alert(...)` writes JSON lines to `alerts.jsonl`.

This is a nice demonstration of separating:

- full event history
- higher-signal operator-facing alert stream

14 server/auth.py: Authentication and Post-Login Integrity

This file handles:

- registration
- login
- session validation
- authenticated request verification

14.1 1 register(...)

The register route:

- normalizes the email
- enforces that the email belongs to the local domain
- checks confirm-password match if provided
- rejects duplicate users
- hashes the password with Argon2id
- inserts the user row
- logs the event

This directly enforces one of the assignment requirements: domain-local account management with non-plaintext password storage.

14.2 2 login(...)

The login route:

- normalizes the email
- gets the client IP

- checks whether the account is currently locked
- loads the user row
- verifies the password hash
- records failures or clears them on success
- creates a session token
- creates a session key
- writes a session row with expiry
- returns `AuthResponse`

An interesting simplification here is that `session_id` and `session_token` are currently the same underlying value. That is acceptable in this local prototype but worth mentioning honestly in a lecture or defense.

14.3 3 get_current_user(...)

This validates:

- bearer token present
- session row exists
- session not expired

It also applies rolling expiry by updating `last_seen` and `expires_at`.

14.4 4 verify_authenticated_request(...)

This is one of the most important functions in the whole system.

It verifies that a state-changing request includes:

- `X-Request-Id`
- `X-Session-Id`
- `X-Seq-No`
- `X-Timestamp`
- `X-Nonce`
- `X-Body-Mac`

It then checks:

- the headers are all present

- `X-Session-Id` matches the current session
- sequence number is valid
- timestamp is within the replay window
- the HMAC matches canonical request content
- the sequence number is greater than the last seen one
- the request ID and nonce have not been used before

Only then does it store the guard values and update `last_seq_no`.

14.5 5 Why This Matters

This is how the project proves secure post-login interaction.

Without this function, a stolen bearer token or replayed request might be enough to repeat an action. With it, the server binds each action to:

- a session
- a unique request ID
- a monotonic sequence number
- a nonce
- a timestamp
- a MAC over the exact request body and metadata

This is much stronger than "user is logged in, so accept every POST".

15 server/rate_limit.py: Abuse Prevention

This module implements throttling and lockout logic.

15.1 1 Sliding Window Budgeting

The generic functions are:

- `_window_stats(...)`
- `_record_event(...)`
- `_retry_after(...)`
- `_raise_limited(...)`
- `enforce_budget(...)`

These allow the project to build both request-count and byte-budget limits.

15.2 2 Login Protection

Relevant functions:

- `check_login_lockout(...)`
- `record_login_failure(...)`
- `clear_login_failures(...)`

The logic uses two buckets:

- per-user
- per-user-plus-IP

This helps resist both simple brute-force and repeated targeted guessing from a single client address.

15.3 3 Send and Upload Limits

Relevant functions:

- `enforce_send_limits(...)`
- `enforce_upload_limits(...)`

Send limits are count-based per minute. Upload limits are byte-budget based per minute.

15.4 4 Audit and Alert Integration

When a budget is exceeded, the module writes:

- an audit event
- an alert event
- an HTTP 429 with `Retry-After`

This connects security enforcement to observability.

16 server/attachments.py: Safe Image Attachments

This module handles attachment storage and retrieval.

16.1 1 Allowed Types

The project only accepts:

- `.png`
- `.jpg`
- `.jpeg`

This narrow scope helps reduce attack surface for the MVP.

16.2 2 Content Validation

`_detect_content_type(data)` checks PNG and JPEG magic bytes.

This is better than trusting the filename extension alone.

16.3 3 Deduplicated Blob Storage

`store_attachment_bytes(...)`:

- validates extension
- enforces max size
- validates magic bytes
- hashes the raw bytes with SHA-256
- uses that hash as the blob key
- stores one physical blob per unique hash
- increments `ref_count` if the same bytes already exist
- creates a new logical attachment row every time

This means two users can upload the same image content and the system stores:

- multiple attachment records
- one shared physical blob

That is a good example of storage optimization under security control.

16.4 4 Authorization on Download

The download route checks whether the current user either:

- created the attachment
- or has a mailbox item linked to it

That prevents ID-based attachment theft.

16.5 5 Cross-Domain Attachment Transfer

For relay, attachments are re-encoded to base64 payloads and transferred through the relay request. On the receiving side, they are stored again using the same validation and dedup pipeline.

17 server/phishing.py: Thin Wrapper, Clear Responsibility

`server/phishing.py` is intentionally small. It wraps the phishing heuristics in `common/text_features.py`.

This is a subtle but good design choice. It means the server can speak in terms of "analyze a message for phishing" while the shared text logic stays reusable.

18 server/mailbox.py: Main Business Logic

This is the largest and most important domain-logic module.

If `storage.py` is the persistent heart of the system, `mailbox.py` is the behavioral heart.

It handles:

- writing mailbox copies
- send dispatch
- delivery-state refresh
- draft management
- read-state updates
- recall
- groups
- quick actions
- TODO creation
- search
- contact autocomplete

18.1 1 Helper Functions

`_touch_contacts(...)` Updates the contact history graph for autocomplete and search.

`_dedupe_emails(...)` Normalizes and deduplicates recipients.

`_build_actions(...)` Builds signed quick-action tokens for inbox messages.

`_mail_row_to_summary(...)` Converts a database row into a `MailSummary` object.

18.2 2 store_mail_copy(...)

This function is extremely important because it defines what a stored mailbox copy contains.

When a message is stored, the function computes and stores:

- keywords
- classification
- phishing/security flags
- quick reply suggestions for inbox mail
- quick actions for inbox mail
- read state
- recall state
- attachment links
- contact updates

This means smart features are not treated as a separate afterthought. They are attached at the mailbox-item level.

18.3 3 Suspicious Mail Handling

If `analyze_message(...)` returns `suspicious=True`, the message classification is forced to `Suspicious`.

Additionally, when the message is stored in an inbox, a `suspicious_mail_detected` event is logged.

That ties intelligent detection to the alerting model.

18.4 4 Sender-Side Delivery State

`refresh_sent_delivery_state(...)` inspects the queue and updates the sender's sent-row status to one of:

- delivered
- queued
- recalled
- partial
- failed

This is a nice teaching point because it shows how mailbox UI state can be derived from background job state.

18.5 5 dispatch_message(...)

This is the send pipeline.

It:

- normalizes recipients and CCs
- loads/export attachments
- creates message ID and thread ID
- splits recipients into local and remote groups
- validates local recipients exist
- validates remote domains are known peers
- stores the sender's sent copy with `delivery_state="queued"`
- enqueues `local_delivery` and `remote_delivery` jobs
- logs `mail_sent`
- returns a queued response

This function is a key demonstration of queue-first architecture.

18.6 6 Draft Handling

The draft route can:

- save a draft
- update an existing draft
- optionally send immediately from a draft

This satisfies the assignment requirement for a drafts folder instead of only send-or-nothing behavior.

18.7 7 Recall Logic

The recall route enforces several checks:

- the authenticated user must own the sent message
- the message must be in the sent folder
- the recall window must not be expired
- local pending jobs can be cancelled
- unread delivered inbox copies can be marked recalled
- remote recipients are handled through relay recall

This is more robust than simply flipping a boolean in the sender's mailbox.

18.8 8 Group Operations

The module implements:

- group create
- group member add
- send to saved group

Groups are stored per owner in `groups_store`, not globally.

18.9 9 Quick Actions

The available actions are:

- `add_todo`
- `acknowledge`

The server verifies the signed token before applying the action. This is exactly the kind of "useful but safe" interactive mail feature the assignment asked for.

18.10 10 Search and Autocomplete

Search works by scoring mailbox content and contact emails using `fuzzy_score`.

Autocomplete uses the same fuzzy approach over the contact store.

This is local, understandable, and sufficient for the prototype scope.

19 server/relay.py: Trusted Cross-Domain Delivery

This file handles the distributed part of the assignment.

Two routes are implemented:

- `/v1/relay/incoming`
- `/v1/relay/recall`

19.1 1 Relay Verification

`_verify_relay_request(...)` checks:

- the source domain is trusted
- required relay headers exist
- the timestamp is valid and recent

- the relay MAC matches canonical content
- the relay nonce has not already been used

This prevents:

- forged relay requests
- replayed relay delivery
- duplicate inbound processing

19.2 2 Incoming Relay

The incoming relay handler:

- checks the claimed relay domain matches the body
- verifies relay security
- ensures all recipients belong to the local domain
- ensures recipients exist
- enqueues an `inbound_delivery` job
- logs the event

Notice that it does not directly store the inbox row inline. That work is queued for the inbound worker.

19.3 3 Relay Recall

The recall relay handler:

- verifies relay security
- cancels pending inbound jobs where possible
- applies recall to already-inserted unread inbox copies
- logs the result

This design covers both "not delivered yet" and "already inserted but unread" cases.

20 server/workers.py: Background Delivery Runtime

This file makes the concurrent architecture real.

20.1 1 Worker Startup

`start_workers(ctx)` launches:

- `delivery-worker-1`
- `delivery-worker-2`
- `inbound-worker-1`

Each worker has a set of job types it is allowed to claim.

20.2 2 Worker Loop

`_worker_loop(...)`:

- continuously claims eligible jobs
- sleeps if no work is available
- processes jobs
- schedules retries on failure
- completes jobs on success
- refreshes sent delivery state where relevant
- logs all meaningful outcomes

20.3 3 Job Type Routing

`_process_job(...)` dispatches to:

- `_process_local_delivery(...)`
- `_process_remote_delivery(...)`
- `_process_inbound_delivery(...)`

This keeps the queue engine generic while job behavior stays readable.

20.4 4 Local Delivery

Local delivery stores inbox copies for users in the same domain.

20.5 5 Remote Delivery

Remote delivery sends a relay request to the peer domain.

20.6 6 Inbound Delivery

Inbound delivery:

- stores relay attachments locally
- inserts inbox copies for local recipients

20.7 7 Why This File Matters So Much

Without this file, the queue design would be aspirational. With this file, the system actually behaves like a small asynchronous distributed system.

21 server/logging.py: Structured Audit and Alerts

This module is intentionally small, but its role is important.

`log_event(...)` always writes an audit event. For certain event types, it also writes an alert.

Alert-producing event types currently include:

- `job_failed`
- `login_lockout`
- `send_rate_limited`
- `upload_rate_limited`
- `login_rate_limited`
- `relay_replay_rejected`
- `request_replay_rejected`
- `request_mac_failed`
- `relay_mac_failed`
- `suspicious_mail_detected`

This gives the project a clean concept of:

- everything that happened
- the smaller subset that should draw attention

22 server/web.py: Serving the Browser Frontend

This file serves the web client.

Routes:

- `/` -> `index.html`
- `/static/...` -> frontend assets
- `/api-info` -> JSON endpoint overview
- `/favicon.ico` -> empty 204

Why it matters:

- users can test the system from a browser
- the browser is a real client, not just a static page
- the API remains the same secure backend underneath

23 Client Layer: client/

The `client/` package gives the project a CLI frontend.

This is useful for:

- testing
- automation
- demonstrations
- comparing browser and script behavior

24 client/api.py: Session Store and Signed API Calls

This file is the CLI-side mirror of the server's authenticated request model.

24.1 1 SessionState

Stores:

- email
- base URL
- session ID
- session token
- session key
- sequence number

24.2 2 SessionStore

Stores per-user session JSON files under `.client_state/`.

This lets the CLI remember login state across commands.

24.3 3 ApiClient.register(...)

Sends registration with confirm-password support.

24.4 4 ApiClient.login(...)

Logs in, receives session info, saves local session state.

24.5 5 _auth_headers(...)

This is the CLI counterpart to `verify_authenticated_request(...)`.

It:

- increments sequence number
- generates request ID
- generates nonce
- computes current timestamp
- builds canonical JSON
- computes HMAC
- attaches all auth headers
- saves updated session state

This is a very good example of client and server sharing one protocol.

24.6 6 API Convenience Methods

The rest of the class maps user actions to routes:

- inbox, sent, drafts
- message inspection
- upload
- send
- save draft
- mark read

- recall
- group create/add/send
- execute action
- todos
- search
- autocomplete

25 client/cli.py, client/ui.py, and client/quick_reply.py

These files round out the CLI layer.

`client/cli.py` Defines command-line arguments and dispatches them to `ApiClient`.

`client/ui.py` Formats output for CLI display.

`client/quick_reply.py` Chooses reply text from either explicit user text or generated suggestions.

Together, these files keep the CLI code cleaner than if everything lived in one large script.

26 Browser Frontend: web/index.html

The browser UI is not just decorative. It exposes nearly all core features.

The main UI areas are:

- registration form
- login form
- compose form
- attachment upload
- search form
- group creation
- group send
- mailbox tabs
- detail view

It also displays:

- current domain
- current user
- sequence number

This helps the user see the security/session state at a glance.

27 Browser Frontend Logic: web/app.js

This file is the browser equivalent of the CLI client plus extra UI behavior.

27.1 1 State Model

The global `state` object stores:

- current domain
- current session
- active mailbox tab
- mailbox data
- todos
- search contacts
- selected message
- compose attachments
- attachment preview

27.2 2 Session Persistence

The browser stores session data in `localStorage`.

27.3 3 Register and Login

`handleRegister()`:

- checks email, password, confirm password
- validates password match client-side
- calls `/v1/auth/register`

`handleLogin()`:

- logs in
- stores session token and session key
- resets sequence number to 0
- refreshes mailbox data

27.4 4 Signed Browser Requests

`buildSignedHeaders(path,body)`:

- increments `seq_no`
- generates request ID and nonce
- computes timestamp
- builds canonical JSON
- signs it with Web Crypto HMAC-SHA256

This means the browser and CLI both participate in the same secure request protocol.

27.5 5 Client-Side Anti-Abuse Guard

`runClientGuard(key,action)` adds:

- duplicate-submit blocking while an action is already running
- a short cooldown before the same action can repeat

This is not the primary enforcement layer. The server still makes the real security decision. But it is a useful client-side anti-misfire and light anti-burst measure.

27.6 6 Rendering Safety

The frontend renders mail text using `escapeHtml(...)`.

This is an important security choice. It prevents a message body like:

```
<imgsrc="x"onerror="alert(1)">
```

from executing as code in the browser UI.

27.7 7 Detail View

The detail panel shows:

- message metadata
- phishing score and reasons
- keywords
- attachments
- quick reply buttons
- quick action buttons
- recall or mark-read buttons when appropriate

This is a strong educational UI because it surfaces internal system metadata to the user instead of hiding it.

28 End-to-End Flow: Registration

Let us walk through the actual code path for registration.

28.1 1 Browser or CLI Side

The client sends:

- email
- password
- confirm password

28.2 2 Server Side

`server/auth.py->register(...)`

It:

- normalizes the email
- checks that the email domain matches the local server domain
- checks password confirmation
- checks the user does not already exist
- hashes the password
- inserts the user row
- logs the register event

28.3 3 Storage Side

The `users` table receives:

- email
- password hash
- created timestamp

29 End-to-End Flow: Login

29.1 1 Client Side

The client sends email and password.

29.2 2 Server Side

`server/auth.py->login(...)`

It:

- checks lockout state
- verifies password hash
- records failure or clears prior failures
- issues session token and session key
- stores session row with expiry

29.3 3 Result

The client now has:

- bearer token for session identity
- session key for HMAC request signing

This is the foundation of all later protected POST actions.

30 End-to-End Flow: Signed Authenticated POST

Take `sendmail` as an example.

30.1 1 Client Builds Headers

The client computes:

- request ID
- session ID
- sequence number
- timestamp
- nonce
- HMAC over canonical request content

30.2 2 Server Verifies Them

`server/auth.py->verify_authenticated_request(...)`

The server checks:

- the session exists and is not expired
- the timestamp is fresh
- the HMAC matches
- the request ID and nonce are unused
- the sequence number is strictly increasing

30.3 3 Why This Is Better Than a Plain Session Cookie

This protects against:

- replaying old requests
- duplicating actions accidentally or maliciously
- tampering with the request body
- stale or reordered action reuse

31 End-to-End Flow: Local Mail Send

Suppose Alice on `a.test` sends to Carol on `a.test`.

31.1 1 Request Handling

`server/mailbox.py->send(...)`

This:

- verifies the authenticated request
- enforces send rate limits
- calls `dispatch_message(...)`

31.2 2 Dispatch

`dispatch_message(...)`:

- stores the sender's sent copy
- enqueues a `local_delivery` job
- returns `queued`

31.3 3 Worker Completion

`server/workers.py->_process_local_delivery(...)`

The worker stores Carol's inbox copy.

31.4 4 Delivery State

After the job completes, sender-side sent state becomes `delivered`.

32 End-to-End Flow: Cross-Domain Mail Send

Suppose Alice on `a.test` sends to Bob on `b.test`.

32.1 1 On Domain A

The request path is the same at first:

- verify request
- enforce send limits
- create sent copy
- enqueue `remote_delivery`

32.2 2 Delivery Worker on Domain A

`_process_remote_delivery(...)` calls `ctx.relay_post_sync(...)`.

This sends:

- source domain
- source email
- recipients
- subject and body
- thread and message IDs
- attachments

with relay headers:

- `X-Relay-Domain`
- `X-Relay-Timestamp`
- `X-Relay-Nonce`
- `X-Relay-Mac`

32.3 3 On Domain B

`server/relay.py->relay_incoming(...)`

This verifies:

- trusted source domain
- timestamp
- nonce uniqueness
- HMAC
- recipient ownership by the destination domain

Then it enqueues `inbound_delivery`.

32.4 4 Inbound Worker on Domain B

`_process_inbound_delivery(...)`:

- stores attachments locally
- writes Bob's inbox copy
- logs success

This is the distributed mail path.

33 End-to-End Flow: Recall

Recall is one of the more subtle flows because it must handle race conditions.

33.1 1 Sender Requests Recall

The sender sends a signed recall request.

33.2 2 Local Validation

`server/mailbox.py->recall(...)`

Checks:

- message exists in sender sent folder
- sender owns it
- recall window has not expired

33.3 3 Queue-Aware Recall

The system first tries to cancel pending jobs.

If mail is still queued and not delivered yet, cancelling the job is enough for that recipient.

33.4 4 Delivered Recall

For already-delivered local inbox copies, `apply_recall(...)` checks:

- not already recalled
- not already read
- still inside recall window

Only then does it mark the mail recalled.

33.5 5 Remote Recall

For cross-domain recipients, the sender's server sends relay recall to the peer.

33.6 6 Why This Design Is Good

It avoids a common prototype mistake: pretending recall is always possible by blindly updating sender-side state only.

34 End-to-End Flow: Quick Actions

Quick actions show how the project treats convenience features safely.

34.1 1 Token Creation

When inbox mail is stored, `_build_actions(...)` creates signed action tokens.

34.2 2 Token Execution

The client sends the token to `/v1/actions/execute`.

The server:

- verifies the request HMAC
- verifies the action token signature
- verifies that the token belongs to the current recipient
- verifies that the action type is allowed

- performs the action

This is far safer than trusting raw `message_id` plus `action` input from the browser.

35 End-to-End Flow: Search and Smart Suggestions

When inbox mail is stored, the system already computes:

- keywords
- classification
- phishing flags
- quick replies

Later:

- search scores messages using fuzzy matching
- autocomplete scores prior contacts
- quick reply buttons let the user answer quickly

The important design choice is that all of these features are advisory. None of them automatically mutate privileged state without explicit user action and server validation.

36 Data Model Deep Dive

This section is useful if you need to explain the project in a more database- oriented way.

36.1 1 Users and Sessions

`users` Identity store.

`sessions` Active authenticated state. Also stores the per-session request-signing key and the last accepted sequence number.

This is what makes replay defense possible without storing every past request forever.

36.2 2 Replay Defense Tables

`request_guards` Prevents duplicate authenticated user requests.

`relay_guards` Prevents duplicate relay traffic between servers.

36.3 3 Mail and Attachments

`mail_items` Mailbox-facing message copies.

`attachments` Logical attachment records.

`attachment_blobs` Physical deduplicated content storage.

`mail_attachment_links` Authorization bridge between mailbox items and attachment IDs.

36.4 4 Productivity and Smart UX

`groups_store` Saved groups for group send.

`contacts` History-based autocomplete and search hints.

`todos` Quick action output.

36.5 5 Asynchronous Runtime

`job_queue` Backbone of queued delivery and retry behavior.

37 Browser UI as a Security Teaching Tool

The browser UI is educationally useful because it exposes otherwise invisible state:

- active domain
- logged-in user
- sequence number
- phishing score
- suspicious reasons
- quick replies
- signed quick actions
- delivery state

In a presentation or lecture, this lets you show that the project is not only secure in backend code, but also designed to make security-relevant state inspectable.

38 Testing Philosophy

The project does not treat tests as an afterthought. The tests in `tests/test_secure_mail.py` are written as executable acceptance evidence.

38.1 1 Test Fixture Setup

The `app_pair()` fixture:

- creates temporary isolated storage roots
- creates domain A and domain B configs
- creates two apps
- injects in-memory relay dispatch functions
- wraps both in `TestClient`

This is clever because it tests two-domain behavior without needing real network sockets.

38.2 2 _signed_headers(...)

The tests sign authenticated requests the same way the real clients do.

That means tests are not bypassing security. They are exercising the actual request-integrity mechanism.

38.3 3 _relay(...)

The helper signs relay requests using the test relay secret. This allows the tests to validate real relay authentication logic.

39 Walkthrough of Key Tests

39.1 1 test_login_lockout

Proves that repeated bad passwords trigger temporary lockout and a `Retry-After` response.

39.2 2 test_register_requires_matching_confirmation

Proves confirm-password enforcement is real, not just a frontend hint.

39.3 3 test_web_root_is_served

Proves the web frontend is actually served by the backend.

39.4 4 test_cross_domain_send_attachment_recall_and_tamper

This is one of the best tests in the suite because it covers many features at once:

- upload attachment

- send cross-domain mail
- worker delivery
- sent state becomes delivered
- inbox attachment visibility
- quick action token tamper rejection
- recall behavior

39.5 5 test_phishing_sample_is_flagged

Proves suspicious mail can be labeled as suspicious based on heuristic content.

39.6 6 test_attachment_dedup_reuses_single_blob

Proves deduplication actually happens at the blob layer.

39.7 7 test_cc_recipients_are_delivered

Proves CC is part of delivery behavior, not just metadata.

39.8 8 test_concurrent_multi_client_send_and_receive

This is the main concurrency acceptance test.

It creates multiple senders in parallel and confirms Bob eventually receives all messages.

39.9 9 test_replay_rejected

Proves that repeating the same signed request is rejected.

39.10 10 test_invalid_attachment_rejected

Proves content-type spoofing by filename does not work.

39.11 11 test_send_rate_limit

Proves the server enforces anti-spam/anti-burst rate limiting.

40 Security Model Mapped Directly to Code

This section connects the threat model to implementation.

40.1 1 Password Theft Risk

Mitigation in code:

- `common/crypto.py->hash_password`
- `common/crypto.py->verify_password`

40.2 2 Brute-Force Login

Mitigation in code:

- `server/rate_limit.py->check_login_lockout`
- `server/rate_limit.py->record_login_failure`
- `server/auth.py->login`

40.3 3 Tampered Authenticated Request

Mitigation in code:

- `client/api.py->_auth_headers`
- `web/app.js->buildSignedHeaders`
- `server/auth.py->verify_authenticated_request`

40.4 4 Replay Attack

Mitigation in code:

- `request_guards`
- `relay_guards`
- sequence tracking in `sessions.last_seq_no`

40.5 5 Open Relay or Forged Relay

Mitigation in code:

- `server/relay.py->_verify_relay_request`
- domain trust mapping in `peer_domains`

40.6 6 Attachment Abuse

Mitigation in code:

- extension allowlist
- magic byte validation
- size limit
- generated storage path
- access control on download

40.7 7 Quick Action Forgery

Mitigation in code:

- `sign_payload(...)`
- `verify_signed_payload(...)`
- recipient binding check in `execute_action(...)`

40.8 8 XSS From Malicious Mail

Mitigation in code:

- `web/app.js->escapeHtml`

This point connects directly to `docs/malicious_mail_poc.md`.

41 Document Set and How It Relates to the Code

This repository already contains a strong document set. Here is how each one fits into the codebase.

41.1 1 docs/protocol.md

Explains the wire protocol:

- auth endpoints
- signed request headers
- send/draft/recall shapes
- relay headers
- lifecycle states

Use this when explaining client/server or server/server communication.

41.2 2 docs/threat_model.md

Explains:

- assets
- trust boundaries
- threats
- mitigations
- residual risk

Use this when explaining the security story at the design level.

41.3 3 docs/test_report.md

Explains:

- test goals
- executed command
- current result
- known gaps

Use this to defend the "testable and reproducible" requirement.

41.4 4 docs/assignment_coverage.md

Maps assignment lines to concrete implementation files. This is excellent for a grader or examiner.

41.5 5 docs/implementation_structure.txt

Explains repository structure and module purpose in a shorter overview form.

41.6 6 docs/alerting_and_audit.md

Explains the logging model:

- audit stream
- alert stream
- event examples

41.7 7 docs/end_to_end_encryption_design.md

This is a design extension document. It explains how the system could evolve to client-side end-to-end encryption.

41.8 8 docs/malicious_mail_poc.md

Explains the risk of unsafe HTML rendering in email clients and shows why the current browser rendering is safer.

41.9 9 docs/p2p_feasibility.md

Explores whether the project should become P2P and concludes that a hybrid approach would be more realistic than pure P2P for this system.

42 Bonus and Extension Topics

The repository includes bonus-topic documentation even when the runtime code does not fully implement the extension.

42.1 1 End-to-End Encryption

The existing architecture could evolve toward:

- public/private key pairs per user
- hybrid encryption per message
- ciphertext storage and relay

But then:

- server-side search
- server-side phishing analysis
- server-side quick replies

would need to move client-side or become metadata-limited.

42.2 2 Malicious Mail Scripts

The code already takes a defensive stance:

- no raw HTML rendering of email body
- escaped text output in the browser
- advisory-only smart features

This is the right posture for hostile-content handling.

42.3 3 P2P Feasibility

The docs correctly note that pure P2P mail introduces major challenges:

- offline availability
- NAT traversal
- spam resistance
- metadata leakage
- weaker auditability

So P2P remains a design study, not the runtime model.

43 What Is Especially Good About This Codebase

This section is useful if you need to defend or evaluate the project.

43.1 1 It Is Small but Real

The system is small enough to read, but it still contains:

- distributed delivery
- storage isolation
- background workers
- replay defense
- attachment security
- testing

43.2 2 Security Is Not Bolted On

Security is visible in:

- schemas
- login logic
- session model
- request MACs
- relay MACs
- attachment validation
- token signing
- audit and alert logging
- frontend escaping

43.3 3 The Smart Features Are Bounded

The project does not pretend intelligence is magic. The smart layer is:

- local
- understandable
- reproducible
- advisory

That is a healthy design choice.

43.4 4 The Tests Prove Real Claims

The tests cover both happy paths and negative/security paths. That makes the project much stronger than a demo that only works on stage.

44 Honest Limitations

No serious educational lecture should hide limitations.

44.1 1 TLS in Demo Mode

The architecture expects HTTPS/TLS, but the local demo often uses plain local HTTP or unverified TLS.

44.2 2 SQLite Scope

SQLite is appropriate here, but a production distributed mail platform would need stronger operational guarantees and probably a different persistence model.

44.3 3 Heuristic Intelligence

Phishing detection, classification, and quick replies are simple heuristics, not advanced ML or LLM systems.

44.4 4 Session Simplification

`session_id` and `session_token` are currently the same underlying value. This is acceptable for the prototype but worth improving in a stronger design.

44.5 5 Queue Reliability Scope

The queue already supports retries and persistence, but it does not yet include:

- dead-letter queues
- rich monitoring dashboard
- advanced worker health reporting

45 How to Explain This Project in an Exam or Presentation

If someone asks, "What did you actually program?", a strong answer is:

I built a local secure email prototype with two isolated domains. Each domain is a FastAPI server with its own SQLite database, blob storage, logs, and worker threads. Users can register, log in, send mail, save drafts, upload image attachments, recall unread mail, send to groups, search mail, use quick reply suggestions, and trigger safe quick actions. After login, all important POST requests are protected by per-session HMAC signing with request ID, timestamp, nonce, and sequence number. Cross-domain delivery is handled through authenticated relay endpoints and a persistent job queue. The system also implements brute-force protection, send and upload rate limiting, phishing heuristics, attachment deduplication, structured audit logs, structured alerts, and automated tests.

That answer is accurate, concise, and grounded in the code.

46 Suggested Reading Path for a New Developer

If a new developer joins the project, tell them to do this:

1. Read `README.md` for the story.
2. Read `docs/assignment_coverage.md` for the requirement mapping.
3. Read `server/main.py` to see how the app is assembled.
4. Read `server/storage.py` to understand the state model.
5. Read `server/auth.py` to understand trust and sessions.
6. Read `server/mailbox.py` to understand mailbox behavior.
7. Read `server/relay.py` and `server/workers.py` to understand distribution.
8. Read `web/app.js` to understand the browser client.
9. Run `tests/test_secure_mail.py` and study the flows.

This sequence reduces confusion quickly.

47 Study Questions

These questions can be used for revision, presentation prep, or a classroom discussion.

1. Why is per-domain `data_root` a security property and not just a deployment

detail?

1. Why does the project use both bearer sessions and per-request HMAC?
2. What problem is solved by `request_guards` beyond normal authentication?
3. Why are quick-action tokens signed instead of trusting raw action names?
4. Why does the send route enqueue work instead of delivering inline?
5. How does the project distinguish logical attachment records from physical

blob storage?

1. Why is escaping email body text in the browser important?
2. What would break if end-to-end encryption were added without redesigning the

smart features?

1. Why is `mail_items` owner-scoped instead of using a single global message

table only?

1. What parts of the system are strongest as an educational prototype, and what

parts would need major work for production use?

48 Final Summary

This codebase implements a complete educational secure email prototype, not just a conceptual design.

At the code level, it demonstrates:

- modular architecture
- isolated domain configuration
- structured data contracts
- password hashing
- authenticated session handling
- post-login request integrity protection
- rate limiting and lockout
- safe attachment handling and deduplication
- mailbox domain logic
- cross-domain relay

- queue-backed workers
- browser and CLI clients
- search and lightweight intelligent features
- audit logging and alert logging
- reproducible tests

At the documentation level, it demonstrates:

- protocol clarity
- threat modeling
- assignment coverage mapping
- reproducible testing
- bonus-topic reasoning

At the educational level, the strongest lesson is this:

security, concurrency, and usability are easier to explain and verify when the codebase is structured around clear boundaries.

That is exactly what this project tries to show.